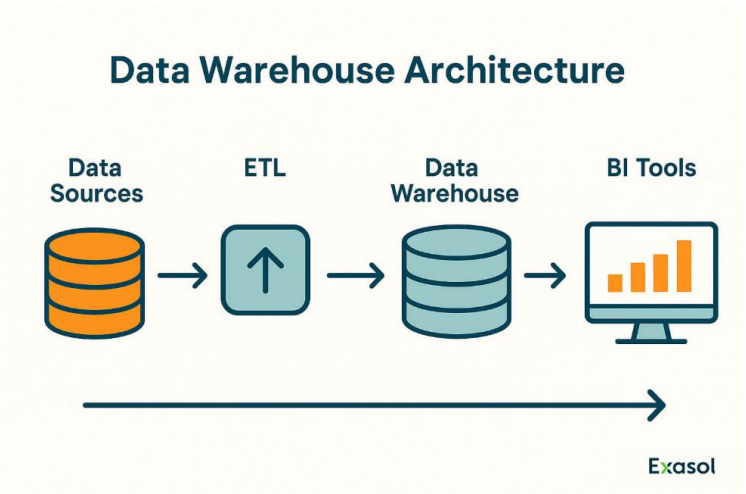
Data Export Methods

E/21/327
Induka Rathnayake

Time series data export is the process of extracting sequential data points (ordered by time/timestamps) from a database or monitoring platform into an external file or system. It enables offline analysis, historical archiving, and feeding data into machine learning models for forecasting or anomaly detection



[Source - https://www.exasol.com/hub/data-warehouse/architecture/]

1. Foundational Data Export Methods (The Basics) :-)

At the foundational level, exporting data means saving numbers from active memory into a permanent file. The choice of file type affects how fast the data can be saved and how much storage space it takes.

1.1 Text-Based Export

CSV

CSV is the most basic and human-readable format. Each row represents a specific moment in time (a timestamp) and each column represents a measurement.

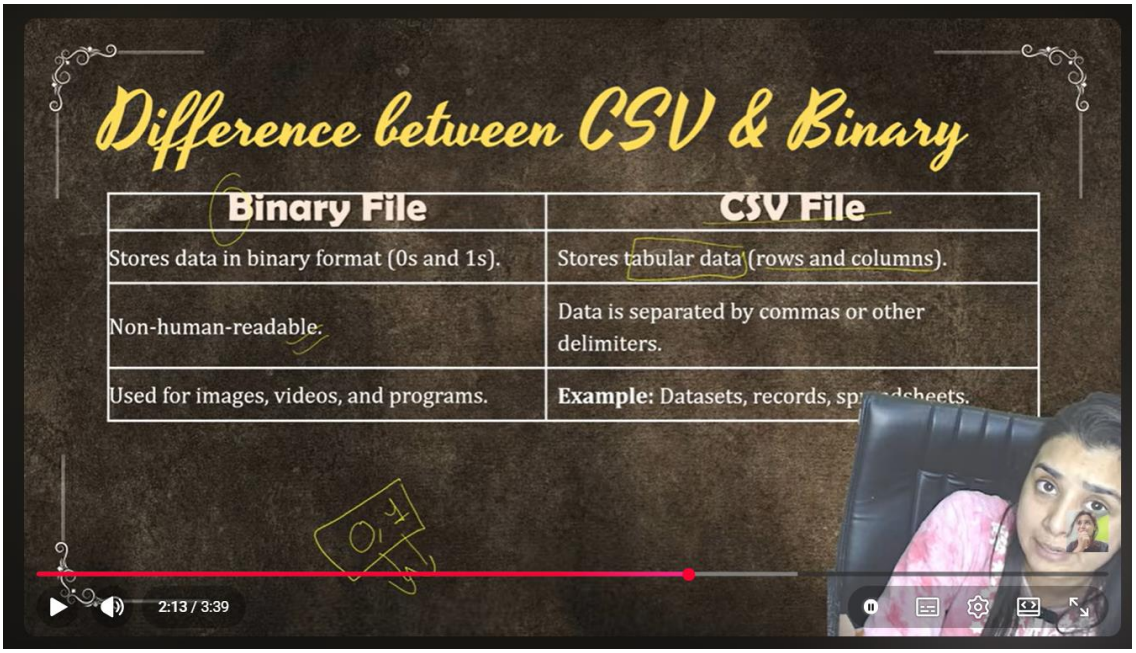
- How it works: System converts numerical values into text strings and separates them with commas.
  - When to use it - Best for very small datasets or simple troubleshooting where you need to open the file in Excel to check values manually.
  - Limitations - Converting raw electrical measurements into text takes unnecessary processing power. For high-frequency transient data, CSV files become massively bloated and slow to load.

1.2 Raw Binary Serialization

.bin, .dat

Instead of converting numbers to text, binary export writes the exact raw bytes directly from the system's memory to the storage drive.

- How it works- A 16-bit integer is simply saved as two bytes, skipping any text conversion.
  - When to use it- This is essential for embedded hardware logging. To capture fast fault transients accurately without overloading, device must stream these raw integer buffers directly to local memory using binary format rather than heavy text strings.



[Source - https://youtu.be/ed5l3C3rsqU?si=VTpCbmPXxp2FyJRL]

2. Modern Data Engineering Paradigms (Intermediate) :-)

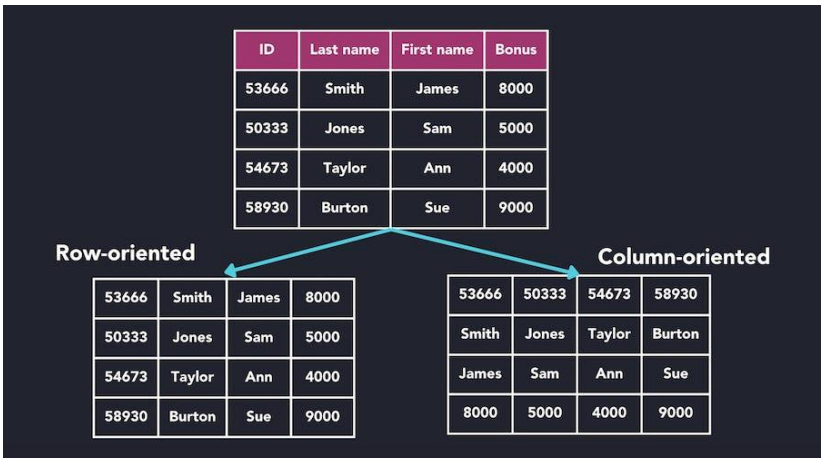
When moving beyond simple files into large-scale machine learning, traditional row-by-row saving becomes a bottleneck. Modern pipelines use columnar formats and streaming networks.

2.1 **High-Performance Data Frame Storage: Format Benchmarking**

Parquet

Standard databases store data row-by-row. Parquet stores data column-by-column.

- Benefit for Deep Learning- Can load specific columns (e.g., just the zero-sequence currents) into memory instantly accelerating the training phase of deep neural networks significantly. Parquet also compresses the data heavily.



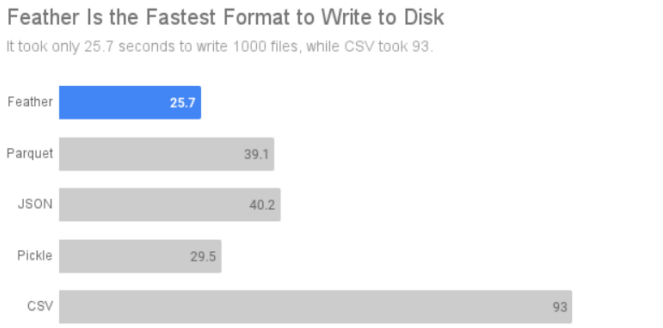
[Source - https://levelup.gitconnected.com/row-database-vs-column-database-how-data-storage-impacts-performance-3cb8b7d633c4]

When handling massive time-series datasets from power system simulations, the read and write speeds of the chosen format become a critical bottleneck. Benchmarking tests reveal massive performance disparities between traditional text formats and modern binary or columnar formats.

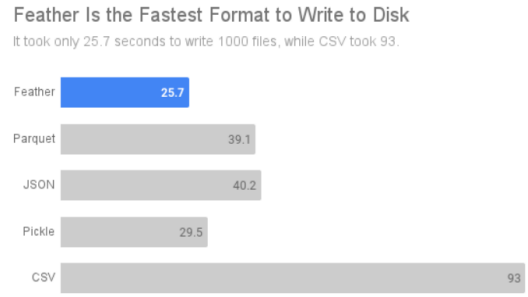
Apache Feather Format

Feather is an optimized binary columnar format designed specifically for fast read/write operations of data frames like Pandas.

- Read Performance - Reading Feather files is significantly faster more than 2X than CSVs. In benchmark testing, feather completed read operations in 15.3 seconds, while CSV lagged far behind at 39.5 seconds. JSON proved to be the least efficient, taking 104 seconds to read.
- Write Performance - Feather is also the fastest format for writing data to disk. Writing 1000 files took Feather only 25.7 seconds, whereas CSV took a massive 93 seconds.



File read/write performance compared between different file formats to store datasets – Image by the author.



[Source - https://towardsdatascience.com/best-file-format-to-store-large-data-dfa47701929f/]

Compare Alternatives (Summary)

While Feather excels in raw speed, other formats offer different trade-offs:

- **Parquet:** Highly compressed and optimized for analytical queries. It recorded a 25.1-second read time and a 39.1-second write time. While slightly slower than Feather, its heavy compression makes it better for long-term cold storage of fault data.
- **Pickle:** Python's native serialization format performs adequately (23.1s read, 29.5s write), but it is restricted to Python environments and lacks cross-language compatibility.
- **JSON & CSV:** Both are human-readable but highly inefficient for machine learning ingestion. JSON took 40.2 seconds to write, while CSV took 93 seconds. Use these only for simple troubleshooting, never for high-frequency transient data pipelines.

Feather is not suited for our FYP because we use Python to train deep learning models on massive amounts of historical fault data. Feather is incredibly fast for moving raw dataframes into RAM. But, it completely lacks compression. If trying to save thousands of high-frequency

transient simulations using Feather, will instantly run out of hard drive space. Parquet solves this by heavily compressing the data while still allowing modern Python DL libraries (like PyTorch and TensorFlow) to read specific sensor columns lightning-fast without crashing your memory. For building a massive scalable DL pipeline. So, best suite is Parquet!

2.2 Message Streaming (MQTT and Kafka)

In a real-world deployed grid, data cannot be batched into a daily file. it must be evaluated instantly.

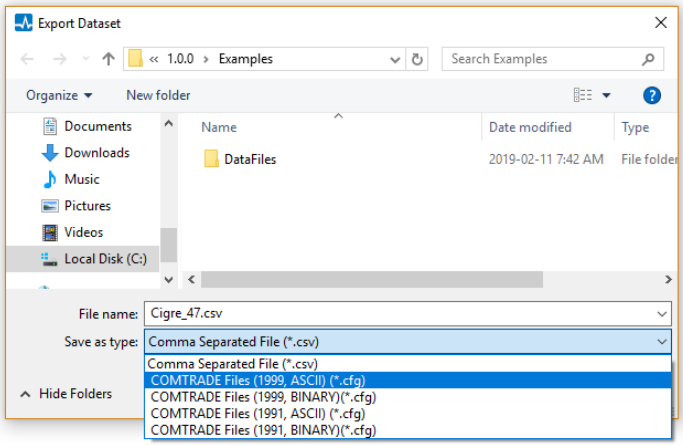
- MQTT: A very lightweight protocol used by low-power field sensors to push data to a server.
- Apache Kafka: A heavy-duty pipeline that can handle millions of high-frequency samples per second organizing them into streams that a deep learning model can read in real time.

3. Power System Simulation Export Protocols :-)

3.1 PSCAD / EMTDC (High-Frequency Transients)

PSCAD simulates complex, high-frequency electromagnetic transients like lightning strikes or sudden short circuits in microsecond time steps.

- COMTRADE Standard - PSCAD natively exports to the IEEE COMTRADE standard. This generates a configuration file (.CFG) and a binary data file (.DAT).
- Integration - Python scripts must be written to parse these COMTRADE files to extract the matrices needed for neural network training.



[Source-https://www.pscad.com/enerplot/Enerplot/Features_and_Operations/Data/Saving_Datasets.htm]

3.2 MATLAB and Simulink

MATLAB provides a flexible environment where the mathematical simulation and the data science environment are closely linked.

- Workspace Logging - Simulation outputs are saved directly into RAM as 'Timeseries' objects.
- Exporting - These objects can be written directly to Parquet files or passed instantly into Python via the MATLAB Engine API.

3.3 OpenDSS & pandapower (Steady State and QSTS)

These are best suite for slower, macro-level grid simulations.

- ✓ pandapower: Built purely on Python, so results are naturally output as Pandas DataFrames.
- ✓ OpenDSS: Uses Monitors placed on network lines to record measurements, which can be exported as ~~CSVs~~ or accessed ~~directly via a Python COM~~ interface.

Summary Matrix of Software Export Protocols

Software Platform	Simulation Capability	Native Export Format	Storage Performance Profile	Deep Learning Pipeline Fit & FYP Suitability
PSCAD	Electromagnetic Transients (EMT)	COMTRADE (.cfg, .dat)	**High (Binary)** Writes transient microsecond data as raw binary bytes, avoiding the severe 93-second write-delays associated with text formats.	**Excellent** Generates the exact high-frequency fault transients required. Requires a Python script to parse the binary data into matrices.
MATLAB / Simulink	Dynamic / EMT / Control Logic	Arrays / Direct to **Parquet**	**High (Columnar)** Can natively write simulation outputs to Apache Parquet, taking advantage of heavy compression and fast 25.1-second read times.	**High** Capable of transient simulation with an optimized, highly efficient data extraction pipeline straight to PyTorch/TensorFlow.
OpenDSS	Quasi-Static Time-Series (QSTS)	**CSV** (via Monitors)	**Poor (Text Bottleneck)** Relies natively on CSV exports. Benchmarks prove CSV is 2X slower to read and over 3.5X slower to write, creating a massive I/O bottleneck for large datasets.	**Moderate to Low** To be viable for deep learning, you must completely bypass its native CSV export and pull data directly into Python memory via the COM interface.
pandapower	Steady-State Power Flow	Pandas DataFrames / Direct to **Feather**	**Optimal (Memory/Binary)** Outputs DataFrames that can be instantly saved as Feather files, yielding the fastest read (15.3s) and write (25.7s) speeds possible.	**Low for FYP** While its export format is mathematically the fastest, it only simulates steady-state grids, which cannot produce the transient waveforms needed for relay fault detection.